UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 05

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: HTML5, XML, CSS3, Flexbox/Grid, Diseño Responsivo y SEO
Modalidad: Presencial / Laboratorio

INTEGRANTES: _____

OBJETIVO DE LA PRÁCTICA

Deben desarrollar un **Panel de Métricas Académicas** que cumpla:

1. **Arquitectura CSR:** index.html mínimo, JS hidrata el DOM. Verificar con Ctrl+U que el HTML inicial no contiene métricas renderizadas.
2. **Componentes anidados:** <App> → <Dashboard> → <MetricCard> → <StatusBadge>. Cada nivel usa props para datos y children para composición visual.
3. **Estilado estratégico:** Usar **CSS Modules** para tarjetas, **inline styles** solo para valores dinámicos (ej. color de progreso), y **framework CSS** opcional para layout global.
4. **Validación de tipos:** JSDoc o PropTypes para documentar la interfaz de cada prop.
5. **Reporte CSR:** Captura de Performance y React DevTools explicando tiempo de hidratación vs render inicial.

Roles Técnicos

Rol	Responsabilidad
Arquitecto de Proyecto	Scaffold Vite, estructura de carpetas, vite.config.js, rutas de importación
Desarrollador de Componentes	JSX, props, children, flujo unidireccional, validación de tipos
Ingeniero de Estilos	CSS Modules, scope, colisiones, transiciones, consistencia visual
QA & CSR Validator	React DevTools, inspección de HTML inicial, reporte de hidratación, documentación

Entregable

- Repositorio GitHub o ZIP con: package.json, src/, index.html, README.md
- README.md debe incluir:

Diagrama de árbol de componentes y flujo de props **Rúbrica de Evaluación**

  - /children
  - Comparativa de estrategias de estilado aplicadas
  - Capturas de React DevTools + explicación de CSR vs HTML estático

Rúbrica de Evaluación

Criterio	Peso
Scaffold correcto, dependencias y estructura Vite/React	15%
Implementación de props y children con flujo unidireccional	30%
Estrategia de estilado coherente (CSS Modules + scope correcto)	20%
Validación CSR (hidratación, DevTools, análisis de HTML inicial)	20%
Documentación técnica, tipos JSDoc y trabajo en equipo	15%

}

CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar)

package.json

```
json
```

```
1  {
2    "name": "is093a-semana05",
3    "private": true,
4    "version": "1.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "dependencies": {
12     "react": "^18.3.1",
13     "react-dom": "^18.3.1"
14   },
15   "devDependencies": {
16     "@vitejs/plugin-react": "^4.3.0",
17     "vite": "^5.4.0"
18   }
19 }
```

src/main.jsx

```
jsx
```

```
1  import React from 'react'
2  import ReactDOM from 'react-dom/client'
3  import App from './App'
4  import './index.css'
5
6  ReactDOM.createRoot(document.getElementById('root')).rende
7    <React.StrictMode>
8      <App />
9    </React.StrictMode>
10 )
```

src/App.jsx

```jsx

/**
 * TODO: Estructurar App como orquestador.
 * Pasar datos de métricas a Dashboard vía props.
 * Usar children para inyectar contenido adicional (ej. t
 */
const App = () => {
  const metricsData = [
    { id: 1, label: 'Asistencia', value: 87, unit: '%', tr
    { id: 2, label: 'Rendimiento', value: 92, unit: '%', t
    // TODO: Añadir al menos 2 métricas más
  ]

  return (
    <main className="app-container">
      {/* TODO: Renderizar Dashboard pasando metricsData y
      <h1>Panel de Métricas UNCP</h1>
    </main>
  )
}

export default App
```

src/components/Dashboard.jsx

```jsx
import MetricCard from './MetricCard'

/**
 * TODO: Recibir props (metrics, title) y children.
 * Renderizar lista de MetricCard iterando metricsData.
 * Usar children para mostrar contenido adicional debajo c
 */
const Dashboard = ({ metrics, children }) => {
  return (
    <section className="dashboard">
      <div className="grid">
        {/* TODO: Iterar y pasar cada métrica como prop a
        {/* Recordatorio: key prop obligatorio en listas '
      </div>
      {children}
    </section>
  )
}

export default Dashboard
```

src/components/MetricCard.jsx

```jsx
import styles from '../styles/MetricCard.module.css'

/**
 * TODO: Recibir { label, value, unit, trend }.
 * Calcular color dinámico inline basado en trend ('up'='g
 * Renderizar estructura semántica con classes de CSS Modu
 */
const MetricCard = ({ label, value, unit, trend }) => {
  return (
    <article className={styles.card}>
      <h3>{label}</h3>
      <div className={styles.valueBox}>
        {/* TODO: Mostrar value + unit */}
        <span className={styles.indicator}></span>
      </div>
    </article>
  )
}

export default MetricCard
```

src/styles/MetricCard.module.css

```
CSS
```

```
.card {
  padding: 1.25rem;
  border-radius: 0.75rem;
  background: #ffffff;
  box-shadow: 0 2px 8px rgba(0,0,0,0.06);
  transition: transform 0.2s ease;
}

.card:hover {
  transform: translateY(-4px);
}

.valueBox {
  display: flex;
  align-items: baseline;
  gap: 0.25rem;
  margin-top: 0.5rem;
}

.indicator {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  background: #cbd5e1;
}
```

NOTAS TÉCNICAS CLAVE

Concepto	Aplicación Práctica
CSR vs SSR	En CSR, index.html solo tiene . React hidrata el DOM tras cargar JS. SEO depende de meta tags estáticos o prerenderizado.
JSX	Sintaxis declarativa que compila a React.createElement(). Requiere cierre de etiquetas y className en lugar de class.

Props & Children	props = datos inmutables descendentes. children = composición de UI (anidación). Flujo unidireccional: padre → hijo.
CSS Modules	Genera hashes únicos (_card_1a2b3). Evita colisiones globales. Ideal para componentes aislados en CSR.
Hidratación	React toma el HTML estático (si existe) o inyecta nodos, luego adjunta event listeners. StrictMode renderiza doble en dev para detectar side effects.
Performance CSR	Optimizar con React.lazy, Suspense, code splitting (vite build genera chunks). Evitar render innecesario con memo/useCallback.

VALIDACIÓN FINAL

1. Verificar que npm run dev levante sin errores en consola.
2. Abrir Ctrl+U y confirmar que el HTML inicial **no contiene métricas renderizadas** (validación CSR).
3. Inspeccionar con **React DevTools**: verificar árbol de componentes, props pasadas y re-renders.
4. Ejecutar npm run build y confirmar que genera dist/ optimizado.
5. Completar README.md con diagrama de flujo, estrategia de estilos y reporte CSR.
6. Subir a GitHub así mismo subir en rar su proyecto en el ADESA.